TUẦN 6 · TRÌNH BÀY VÒNG 1

KIỂM THỬ TỰ ĐỘNG

Tổng quan, kiến trúc **Appium** và demo bộ test **Đăng nhập** trên ứng dụng Android **Swag Labs**.



NHÓM

A1 · KTPM Class 6

CÔNG CỤ

Appium 2 · UiAutomator2 · Python · pytest

Kiểm thử tự động là gì?

> Dùng **công cụ và script** điều khiển ứng dụng, thực hiện các bước kiểm thử và **so sánh kết quả thực tế** với kết quả mong đợi — không cần thao tác tay từng lần.

01

Vì sao nên dùng?

- Chạy lặp lại mỗi khi code thay đổi.
- Kiểm tra hồi quy nhanh, ít bỏ sót.
- Chạy nhiều thiết bị / môi trường.
- Phát hiện lỗi sớm, tiết kiệm thời gian.

02

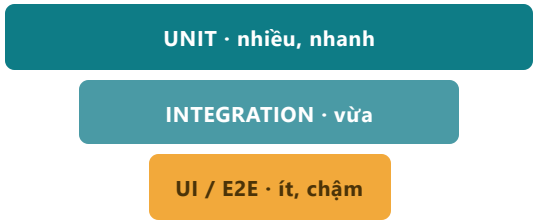
Khi nào không nên?

- Giao diện thay đổi liên tục.
- Dự án ngắn, chỉ chạy một lần.
- Cần đánh giá cảm quan / thẩm mỹ.
- Chi phí bảo trì lớn hơn giá trị mang lại.

03

Đừng chỉ nhìn UI

Kiểm thử giao diện nên **ít và có chọn lọc**: ưu tiên các luồng quan trọng nhất. Phần logic nên kiểm tra ở tầng unit / integration.



Test pyramid: càng lên cao càng chậm, đắt và dễ hỏng — nên dùng cho các luồng người dùng quan trọng, không phải mọi màn hình.

Ba cách làm phổ biến

RECORD & PLAYBACK

Ghi và phát lại

Ưu: nhanh, không cần code, dễ bắt đầu.

Nhược: dễ vỡ khi UI đổi, khó tái sử dụng, khó chạy CI.

Phù hợp: học thử, kiểm tra nhanh một luồng.

Ví dụ: Selenium IDE.

SCRIPT-BASED

Viết bằng code

Ưu: kiểm soát hoàn toàn, tái sử dụng, dễ bảo trì, chạy CI.

Nhược: cần biết lập trình, tốn thời gian ban đầu.

Phù hợp: dự án dài, cần bộ test ổn định.

Ví dụ: Selenium / Appium + pytest.

LOW-CODE / NO-CODE

Công cụ trực quan

Ưu: dễ dùng cho người không chuyên, có sẵn dashboard.

Nhược: phụ thuộc nhà cung cấp, khó tùy biến sâu.

Phù hợp: team nhỏ, nhu cầu đơn giản.

Ví dụ: Katalon, TestProject.

A1

Nhóm A1 chọn Script-based: Python + pytest + Appium 2. Vì cần hiểu sâu cách hoạt động, dễ tách hàm dùng chung, chạy được toàn bộ bộ test bằng một lệnh và sẵn sàng tích hợp CI ở các tuần sau.

Một lệnh Appium đi từ đâu tới đâu?



Connection refused
Appium server chưa chạy / sai cổng 4723.

device not found
Emulator chưa mở hoặc ADB không thấy thiết bị.

element not found
Sai locator hoặc chưa chờ đủ lâu.

app not installed
Sai package hoặc chưa cài APK.

Tìm ô đăng nhập: chuyện gì xảy ra?

```
# 1. Chờ ô Username hiển thị
o_username = WebDriverWait(driver, 30).until(
    EC.visibility_of_element_located(
        (AppiumBy.ACCESSIBILITY_ID,
         "test-Username")
    )
)

# 2. Nhập tên đăng nhập
o_username.clear()
o_username.send_keys("standard_user")

# 3. Bấm LOGIN
driver.find_element(
    AppiumBy.ACCESSIBILITY_ID, "test-LOGIN"
).click()
```

1. **Client sinh lệnh.** `find_element` biến thành yêu cầu W3C: tìm phần tử theo `accessibility id`.
2. **HTTP → Appium server.** Yêu cầu được gửi tới `127.0.0.1:4723`, kèm `sessionId`.
3. **Server chọn driver.** Session đang dùng driver **UiAutomator2**, nên lệnh được chuyển tiếp tới driver này.
4. **Driver gọi ADB / instrumentation.** UiAutomator2 đọc cây giao diện Android để tìm phần tử có id `test-Username`.
5. **Thiết bị trả kết quả.** Element id (hoặc lỗi) quay ngược về client; `WebDriverWait` kiểm tra điều kiện hiển thị.
6. **`send_keys` / `click`.** Một chu trình mới được gửi xuống thiết bị để nhập chữ hoặc bấm nút.

Ba test case Đăng nhập

ID	DỮ LIỆU	KẾT QUẢ MONG ĐỢI	THỰC TẾ
TC-LOGIN-01	`standard_user` `secret_sauce`	Vào được màn hình **PRODUCTS**.	PASSED
TC-LOGIN-02	`locked_out_user` `secret_sauce`	Hiện thông báo **Sorry, this user has been locked out.**	PASSED
TC-LOGIN-03	`standard_user` *(mật khẩu trống)*	Hiện thông báo **Password is required**.	PASSED

Mỗi test mở một session mới; test 1 tự đăng xuất trong `finally` để trả app về màn Login.

3/3 PASSED
khoảng 46 giây

CHẠY TOÀN BỘ

```
pytest tuan-05/test_login.py -v
```

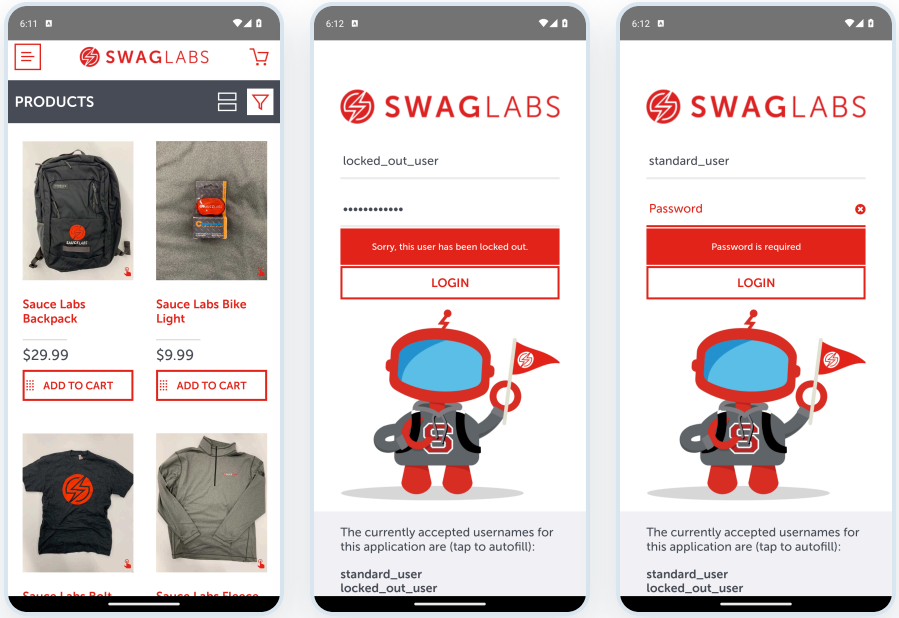
LOCATOR DÙNG TRONG DEMO

`test-Username` `test-Password` `test-LOGIN` `test-PRODUCTS` `test-Error message` `test-Menu` `test-LOGOUT`

ĐIỂM ĐÁNG CHÚ Ý

Khung lỗi là **ViewGroup** nên phải lấy **TextView con** mới đọc được nội dung.

Chạy thật, nhìn thật, sửa thật



01 · Đăng nhập đúng vào PRODUCTS

02 · Tài khoản bị khoá

03 · Bỏ trống mật khẩu

Ảnh chụp từ emulator · bộ test `tuan-05/test_login.py`

01 **Accessibility id ổn định hơn** XPath dài hay tọa độ, vì gắn với phần tử chứ không gắn với vị trí màn hình.

02 `WebDriverWait` **thay** `time.sleep`**.** Chờ đúng điều kiện: hiển thị, bấm được — test nhanh và ít flaky hơn.

03 **Đọc đúng cây giao diện.** `test-Error message` là `ViewGroup`; nội dung nằm trong `TextView` con.

04 **Phân biệt lỗi môi trường và lỗi test.** Server, emulator, ADB, APK — kiểm tra trước khi sửa locator.

05 **Dọn trạng thái trước mỗi test.** Đăng xuất / reset app để bài sau không phụ thuộc bài trước.

07 · TỔNG KẾT

Hiểu kiến trúc trước, viết test sau

Ba điều nhóm A1 mang theo sang tuần 7.

01

Biết lỗi ở tầng nào

Client → server → driver → ADB → thiết bị. Mỗi tầng có một kiểu lỗi riêng.

02

Chọn locator ổn định

Ưu tiên accessibility id; không phụ thuộc tọa độ hay XPath tuyệt đối.

03

Chờ đúng điều kiện

Dùng WebDriverWait với điều kiện rõ ràng; không dùng sleep mò.

TUẦN 7 · CHỨC NĂNG 2

Sắp xếp sản phẩm trên Swag Labs — chọn “Price low to high”, kiểm tra sản phẩm đầu tiên là rẻ nhất; thêm trường hợp sai và thí nghiệm không dùng wait.

Nguồn tham khảo: Appium Documentation (Introduction, UiAutomator2 Driver) · Selenium Documentation (WebDriver, Components) · pytest Documentation (Get started, Assertions) · Martin Fowler – The Practical Test Pyramid.

